

IRPP Project

Team ROSCorp

Report detailing the solution to the problem statement provided for the final course project

of Introduction to Robotics : Perception & Planning

Abhinav P V

Roll No. 2025121015

LCD

abhinav.pv@research.iiit.ac.in

Ahamed Nishan

Roll No. 2024112025

ECD

ahamed.adikool@research.iiit.ac.in

Bejaine Birju

Roll No. 2024102060

ECE

bejaine.birju@students.iiit.ac.in

March 2026

Contents

1	Problem Statement	4
2	Research in Path Planning Algorithms	5
2.1	Overview of Algorithms	5
2.1.1	Artificial Potential Fields (APF)	5
2.1.2	Dynamic Window Approach (DWA)	5
2.1.3	Vector Field Histogram (VFH)	5
2.1.4	Timed Elastic Band (TEB)	6
2.1.5	Dynamic Window Based Controller (DWB)	6
2.1.6	Model Predictive Path Integral (MPPI)	6
2.2	Benchmark Performance	7
2.2.1	Historical Context	7
2.2.2	Static Obstacle Scenario (Unknown Environment)	7
2.2.3	Dynamic Obstacle Scenario	8
2.2.4	Inference	8
3	Algorithms Chosen	9
3.1	Dynamic Window Based Controller (DWB) Algorithm	9
3.1.1	Theoretical Foundation :	9
3.1.2	Justification for Choice :	9
3.2	Model Predictive Path Integral (MPPI) Algorithm	9
3.2.1	Theoretical Foundation :	10
3.2.2	Justification for Choice :	10
4	Algorithm Theory	11
4.1	DWB Algorithm	11
4.1.1	The Search Space (The Dynamic Window)	11
4.1.2	Forward Simulation (Trajectory Rollout)	11
4.1.3	The Objective Function (Critic Scoring)	11
4.2	MPPI Algorithm	12
4.2.1	The Nominal Control Sequence and Noise Injection	12
4.2.2	Stochastic Forward Simulation (Rollouts)	12
4.2.3	Trajectory Cost Evaluation	13
4.2.4	Information Theoretic Control Update	13
5	Simulation Environments	14
5.1	Map 1 : Static Environment	14
5.2	Map 2 : Dynamic Pillar Grid	14
5.3	Map 3 : Urban Cityscape Junction	14
6	Implementation of the Algorithms	16
6.1	DWB Algorithm with VFH	16
6.1.1	Environment and Local Map Representation	16
6.1.2	Vector Field Histogram (VFH) Steering	16

6.1.3	Zone-Based Dynamic Limits	17
6.1.4	Algorithm Pseudocode	17
6.1.5	Modular Critic Implementation and Optimization	18
6.1.6	Fallback Recovery and Visualisation	18
6.2	MPPI Algorithm	18
6.2.1	Environment and Local Map Representation	19
6.2.2	Algorithm Pseudocode and Kinematic Simulation	19
6.2.3	Trajectory Evaluation, Critic Implementation, and Complexity	19
6.2.4	Control Update and Execution	20
6.2.5	ROS 2 Visualisation	21
7	Simulations	22
7.1	DWB	22
7.1.1	Static Environment	22
7.1.2	Dynamic Environment	24
7.1.3	Realistic Environment	26
7.2	MPPI	28
7.2.1	Static Environment	28
7.2.2	Dynamic Environment	30
7.2.3	Realistic Environment	32
8	Performance Analysis	34
8.1	Experimental Setup and Environments	34
8.2	Evaluation Metrics	34
8.3	Observation	35
8.4	Observations and Analysis	35
8.4.1	Static Environment Navigation	35
8.4.2	Dynamic Environment Navigation	36
8.4.3	Computational Load	36
8.5	Final Decree	36

1 Problem Statement

Local planning in an unknown environment

In this project, teams are required to implement a local motion planning algorithm that enables a robot to move toward a desired velocity direction while avoiding obstacles. The teams must demonstrate at least two algorithms for local collision avoidance and justify which performs better and why.

Bonus marks are awarded for demonstrating local planning in dynamic environments (moving obstacles).

Required Platform: Gazebo + TurtleBot3

2 Research in Path Planning Algorithms

A variety of local planning algorithms were evaluated to determine the most suitable approach for the TurtleBot3 platform in Gazebo in both static and dynamic environments : APF, DWA, VFH, TEB, DWB, and MPPI.

2.1 Overview of Algorithms

2.1.1 Artificial Potential Fields (APF)

The environment is modeled as a physical field where the robot is a particle moving under the influence of potential forces. The goal location exerts an *attractive force*, pulling the robot toward it, while obstacles exert *repulsive forces*, pushing the robot away. The robot follows the negative gradient of the total potential function to find its path.

Pros	Cons
Computationally efficient and easy to implement for real time avoidance	High risk of trapping in local minima (areas where forces balance out and the robot stops before reaching the goal, e.g. U-shaped obstacles)
Requires minimal environmental knowledge	Oscillatory behavior in narrow passages or "choke points"

2.1.2 Dynamic Window Approach (DWA)

DWA operates directly in the velocity space (v, ω) . It samples admissible linear and angular velocities within the robot's dynamic constraints (acceleration limits) and forward simulates short term trajectories. It selects the command that maximizes an objective function based on goal alignment, obstacle clearance, and forward velocity.

Pros	Cons
Computationally efficient and widely supported in ROS	Myopic (single step horizon) logic
Directly considers the robot's physical limits and dynamics	Prone to oscillations and getting stuck in local minima

2.1.3 Vector Field Histogram (VFH)

VFH constructs a polar obstacle density histogram from LiDAR scans to represent the environment. It reduces this histogram into a binary representation of "free" and "occupied" sectors. The algorithm then selects a steering direction toward the goal that minimizes cost while passing through a free sector.

Pros	Cons
Highly reactive and robust in unknown or changing environments	Generates non smooth, “jagged” trajectories
Accounts for the robot’s width and sensor uncertainty.	Struggles in narrow corridors due to sector thresholding issues.

2.1.4 Timed Elastic Band (TEB)

TEB formulates local planning as a multi objective optimization problem. Instead of planning a single next velocity command, TEB plans and optimizes a whole sequence of future positions simultaneously, a concept referred to as the “elastic band.” Each position in this sequence is defined as a pose (x, y, θ) , and all poses together form a continuous trajectory. By optimizing all poses at once, the algorithm can effectively deform the trajectory in real time to minimize traversal time and obstacle proximity while strictly respecting non holonomic constraints (the kinematic limits of the robot). This global-within-local approach allows the robot to “anticipate” turns or obstacles further along the local path.

Pros	Cons
Produces time optimal and smooth trajectories	Computationally intensive due to complex optimization
Handles dynamic obstacles well by deforming the trajectory	Very sensitive to parameter tuning (e.g., weight factors)

2.1.5 Dynamic Window Based Controller (DWB)

As the modern successor to DWA in the Nav2 stack, DWB introduces a modular “critic” architecture. It discretizes the velocity search space and evaluates potential trajectories using a series of plugins (critics) that score path adherence, goal progress, and obstacle collision.

Pros	Cons
Highly flexible and tunable through modular critics	Short prediction horizon limits high speed performance
Seamless integration with the ROS2 Nav2 framework	Performance is heavily dependent on precise critic weighting

2.1.6 Model Predictive Path Integral (MPPI)

MPPI is a stochastic model predictive controller that leverages Monte Carlo sampling. It generates thousands of noisy control sequences over a finite time horizon, simulates the resulting trajectories, and updates the optimal control sequence using a cost weighted average.

Pros	Cons
Anticipates dynamic obstacles and produces very smooth control	High computational cost compared to sampling based planners
Extremely robust in complex or highly cluttered environments	Performance scales with sample count, requiring modern CPUs

2.2 Benchmark Performance

We reference benchmark results reported in [1], which evaluates local planners in both static (unknown) and dynamic environments using ROS2.

2.2.1 Historical Context

The chronological evolution of standard local planners primarily follows **VFH** (1991) → **DWA** (1997) → **TEB/E-Band** (2012).

2.2.2 Static Obstacle Scenario (Unknown Environment)

Metric	Simulation			Real-World		
	DWB	MPPI	RPP	DWB	MPPI	RPP
Total Time (s)	29.80	28.35	29.60	24.68	22.92	25.80
Total Distance (m)	12.65	12.45	12.83	9.02	9.12	9.43
Smoothness	0.86	0.89	0.87	0.81	0.86	0.81
Min Dist. to Obstacle (m)	0.19	0.23	0.23	0.19	0.20	0.20
Completed (out of 10)	10	10	10	10	10	10
No. of Recoveries	0	0	0	0	0	2

Table 1: Comparison of simulation and real-world performance in a static unknown environment [1].

2.2.3 Dynamic Obstacle Scenario

Metric	Simulation			Real-World		
	DWB	MPPI	RPP	DWB	MPPI	RPP
Total Time (s)	21.24	22.50	22.79	26.98	27.67	28.95
Total Distance (m)	9.26	9.28	9.41	9.62	9.72	9.65
Smoothness	0.91	0.91	0.91	0.82	0.85	0.85
Min Dist. to Obstacle (m)	0.23	0.35	0.30	0.13	0.16	0.15
Completed (out of 10)	10	10	8	10	10	7
No. of Recoveries	0	0	0	0	0	6

Table 2: Comparison of simulation and real-world performance in a dynamic environment [1].

2.2.4 Inference

According to the benchmark data in [1], it highlights distinct operational trade-offs between the planners:

- **Static Environments:** MPPI is superior. Its predictive horizon efficiently handles static clutter, yielding faster real-world completion (22.92s vs. 24.68s), safer obstacle clearance (0.20m vs. 0.19m), and higher trajectory smoothness (0.86 vs. 0.81).
- **Dynamic Environments:** DWB is faster. Its short-term velocity evaluation enables aggressive reactivity to moving obstacles, completing the real-world course quicker (26.98s vs. 27.67s). However, MPPI maintains a safer clearance margin (0.16m vs. 0.13m).
- **Observation:** We can use MPPI for safety, smoothness, and routing efficiency in dense static spaces & DWB for raw reactive speed in highly dynamic scenarios.

3 Algorithms Chosen

We selected the Dynamic Window Based Controller (DWB) and Model Predictive Path Integral (MPPI) control. DWB represents a mature, highly reactive, sampling based baseline. Its modular critic based structure allows systematic tuning and stable performance. MPPI represents an advanced, multi step, stochastic predictive approach capable of anticipating dynamic changes. Evaluating these two allows us to directly contrast short horizon reactive performance against finite horizon optimization.

3.1 Dynamic Window Based Controller (DWB) Algorithm

DWB is an evolution of the classic Dynamic Window Approach (DWA), modernized for modularity within the ROS 2 Nav2 stack. It is a local reactive planner that computes safe velocity commands by directly evaluating the kinematic capabilities of the robot against the immediate obstacle environment.

3.1.1 Theoretical Foundation :

Instead of calculating a geometric path, DWB operates directly in the velocity space. It discretizes the feasible velocity commands (both linear and angular) into a “dynamic window” of commands that can be reached within a short time step, given the robot’s acceleration limits. It then forward simulates the robot’s kinematics for each velocity pair to predict short trajectories. Each trajectory is scored by a configurable set of heuristic “critics,” and the command yielding the lowest overall cost is sent to the mobile base.

3.1.2 Justification for Choice :

DWB was chosen as our baseline because its modular critic architecture allows systematic tuning. By isolating specific behaviors (like path adherence vs. obstacle clearance), we can comfortably debug its reactions. While it is fundamentally myopic due to its short prediction horizon, it excels in static, highly constrained environments. Furthermore, DWB’s minimal computational overhead ensures it runs at exceptionally high frequencies, making it an ideal candidate for reliable testing on the TurtleBot3 Burger platform within our Gazebo simulation framework.

3.2 Model Predictive Path Integral (MPPI) Algorithm

While DWB reacts to immediate states, MPPI is an information theoretic, sampling based Model Predictive Control (MPC) algorithm. It optimizes a sequence of control inputs over a finite time horizon, making it highly effective at anticipating dynamic obstacles and producing smooth, continuous motion.

3.2.1 Theoretical Foundation :

MPPI avoids the pitfalls of local minima common in gradient based optimization by using Monte Carlo sampling. It samples thousands of noisy control sequences, simulates the robot’s forward dynamics for each sequence, and evaluates the trajectory cost. Instead of picking the single “best” trajectory, MPPI applies a risk sensitive, cost weighted average across all sampled trajectories to update the nominal control sequence. This allows the planner to find pathways through complex, cluttered, and dynamic environments by exploring a wide funnel of future possibilities. Because it evaluates a time extended horizon, the algorithm inherently computes the temporal evolution of the environment, making it naturally suited to foresee and smoothly circumnavigate moving entities rather than abruptly swerving at the last second.

3.2.2 Justification for Choice :

MPPI was chosen to ensure smooth navigation in dynamic environments. Its multi step stochastic rollouts provide the necessary foresight to avoid moving obstacles before they become an immediate collision threat. Although sampling thousands of trajectories per control cycle is computationally heavy, the parallelized nature of MPPI’s tensor operations effectively utilizes multi core architecture.

4 Algorithm Theory

4.1 DWB Algorithm

The Dynamic Window Approach is an entirely local, reactive planner. It doesn't know where the ultimate destination is; it only knows the direction it wants to go right now and what it can see right now.

It operates in three distinct mathematical and computational steps :

4.1.1 The Search Space (The Dynamic Window)

The robot cannot instantly change velocity to any value. It is bound by physical limitations. The algorithm defines a “window” of reachable velocities (v, ω) in the next time step Δt .

Kinematic Constraints (V_s) : The absolute maximum and minimum speeds of the motors.

$$V_s = \{(v, \omega) \mid v \in [v_{min}, v_{max}], \omega \in [\omega_{min}, \omega_{max}]\}$$

Dynamic Constraints (V_d) : What velocities can actually be reached given the robot's current velocity (v_c, ω_c) and maximum accelerations $(\dot{v}_{max}, \dot{\omega}_{max})$ over the control period Δt .

$$V_d = \{(v, \omega) \mid v \in [v_c - \dot{v}_{max}\Delta t, v_c + \dot{v}_{max}\Delta t], \omega \in [\omega_c - \dot{\omega}_{max}\Delta t, \omega_c + \dot{\omega}_{max}\Delta t]\}$$

The Window (V_r) : The valid search space is the intersection of these sets.

$$V_r = V_s \cap V_d$$

4.1.2 Forward Simulation (Trajectory Rollout)

For every valid velocity pair (v, ω) generated by stepping through the Dynamic Window using the resolution parameters, the algorithm predicts where the robot will be over a future horizon T (*predict_time*). It uses the standard unicycle kinematic model :

$$x_{t+1} = x_t + v \cos(\theta_t) \Delta t$$

$$y_{t+1} = y_t + v \sin(\theta_t) \Delta t$$

$$\theta_{t+1} = \theta_t + \omega \Delta t$$

4.1.3 The Objective Function (Critic Scoring)

While classic DWA uses a monolithic, three part cost function, DWB evaluates each simulated trajectory using a modular, weighted sum of independent heuristic functions known as *Critics*. The goal is to minimize the total cost:

$$Cost_{total}(v, \omega) = \sum_{i=1}^N w_i \cdot C_i(v, \omega)$$

where w_i is a configurable tuning weight and C_i is the score returned by a specific critic.

Standard critics integrated into this architecture include:

- **GoalAlign (Heading)** : Measures the angle difference between the end of the predicted trajectory and the desired goal heading vector.
- **ObstacleFootprint (Clearance)** : Measures the distance to the closest obstacle from the simulated pose. Trajectories that intersect obstacles within the robot’s physical radius are assigned an infinite cost to guarantee collision avoidance.
- **BaseProgress (Velocity)** : Rewards trajectories that allow the robot to maintain a higher forward velocity, preventing the robot from becoming overly timid or deadlocking in low velocity states.

4.2 MPPI Algorithm

Unlike the DWB planner, which evaluates a single velocity step based on current bounds, MPPI operates over a rolling time horizon T . It solves a stochastic optimal control problem by leveraging thousands of simulated futures. The algorithm functions through four primary mathematical phases:

4.2.1 The Nominal Control Sequence and Noise Injection

At any given control step, MPPI maintains a “nominal” or baseline control sequence over the horizon T :

$$U = [u_0, u_1, \dots, u_{T-1}]$$

where $u_t = [v_t, \omega_t]^T$ represents the linear and angular velocity commands at time step t .

To explore the environment, MPPI generates K independent sampled control sequences by injecting Gaussian noise into the nominal sequence. For the k -th sample, the noise sequence is:

$$E_k = [\epsilon_0^k, \epsilon_1^k, \dots, \epsilon_{T-1}^k], \quad \epsilon_t^k \sim \mathcal{N}(0, \Sigma)$$

where Σ is the covariance matrix defining the exploration variance for linear and angular velocities. The resulting k -th sampled control sequence is $V_k = U + E_k$.

4.2.2 Stochastic Forward Simulation (Rollouts)

For each of the K sampled control sequences, the algorithm simulates the robot’s kinematic state forward in time. Using the unicycle model from an initial state x_0 , the state at time $t + 1$ for sample k is computed as:

$$x_{t+1}^k = F(x_t^k, u_t + \epsilon_t^k)$$

This results in K distinct predicted trajectories branching out from the robot’s current position.

4.2.3 Trajectory Cost Evaluation

Each predicted trajectory is evaluated to determine its viability. The total state cost $S(V_k)$ for the k -th rollout is the sum of instantaneous state costs $q(x_t^k)$ along the horizon, plus a terminal cost $\phi(x_T^k)$:

$$S(V_k) = \phi(x_T^k) + \sum_{t=1}^T q(x_t^k)$$

The cost function $q(x)$ typically penalizes:

- **Collisions:** Infinite or extremely high cost if the state intersects an obstacle (static or dynamically predicted).
- **Goal Deviation:** Euclidean distance or orientation mismatch relative to the target waypoint or desired velocity vector.

4.2.4 Information Theoretic Control Update

Instead of simply executing the single trajectory with the lowest $S(V_k)$, MPPI uses an information-theoretic approach. It computes a weight w_k for each trajectory using a softmax function parameterized by a "temperature" variable λ :

$$w_k = \frac{\exp\left(-\frac{1}{\lambda}S(V_k)\right)}{\sum_{j=1}^K \exp\left(-\frac{1}{\lambda}S(V_j)\right)}$$

The temperature λ dictates the risk sensitivity. A low λ heavily favors only the absolute best trajectories, whereas a higher λ averages over a broader set of acceptable paths, resulting in smoother behavior.

Finally, the nominal control sequence is updated by taking the cost-weighted average of the noise sequences:

$$u_t \leftarrow u_t + \sum_{k=1}^K w_k \epsilon_t^k$$

The first command of this updated sequence, u_0 , is sent to the robot, the sequence is shifted forward by one time step (discarding u_0 and initializing a new u_{T-1}), and the entire process repeats.

5 Simulation Environments

To comprehensively evaluate the robustness, reactivity, and predictive capabilities of the local path planning algorithms, simulations were conducted across three progressively complex environments in Gazebo. Each map is designed to isolate and test specific navigational challenges, ranging from basic static avoidance to complex, real world dynamic interactions.

5.1 Map 1 : Static Environment

The first environment utilizes the default TurtleBot3 world, which consists of a walled enclosure populated with nine static obstacle columns.

- **Objective :** This map serves as the baseline evaluation for static obstacle avoidance.
- **Testing Focus :** It tests the algorithm’s fundamental ability to navigate tight spaces, maintain safe clearances from stationary objects, and recover from potential local minima (e.g., navigating into dead ends or corners) without the added complexity of moving entities.

5.2 Map 2 : Dynamic Pillar Grid

The second environment introduces a highly constrained, continuously shifting obstacle course. The map features a 3x3 grid of cylindrical pillars. While the outer columns of the grid remain stationary, the center pillar of each row is controlled by an external ROS 2 node (`pillar_mover.py`) to sweep back and forth along the Y axis in a continuous triangle wave pattern.

- **Objective :** This map isolates the algorithms’ ability to handle purely dynamic obstacles in close quarters.
- **Testing Focus :** It stresses the reactivity of the DWB controller and the temporal anticipation of the MPPI algorithm. The staggered phase offsets of the moving pillars create closing gaps, requiring the robot to either wait for a clear path, adjust its speed, or perform evasive swerves to prevent collisions.

5.3 Map 3 : Urban Cityscape Junction

The final environment is a custom built, realistic urban simulation (`city_junction.world`) scaled to match the dimensions of the TurtleBot3. It features a complex layout including a main four way intersection, a T junction branch road, sidewalks, distinct buildings. Furthermore, the environment includes two dynamic cars governed by a state machine (`city_traffic.py`) that constantly moves through the primary East-West and North-South roads.

- **Objective :** This map simulates a real world, unstructured navigational use case.

- **Testing Focus :** The robot must navigate toward its desired heading while contending with a mix of complex static geometry (such as V shaped building corners and narrow sidewalks) and high speed dynamic traffic. It evaluates whether the algorithms can safely cross active traffic lanes and avoid complex static traps in a high clutter environment.

6 Implementation of the Algorithms

6.1 DWB Algorithm with VFH

The implementation of the Dynamic Window-Based Controller (DWB) in our ROS 2 architecture operates as a highly reactive, standalone node (`DWBPlannerNode`). To overcome the classic DWB limitation of oscillating in front of flat walls or falling into local minima, the architecture is augmented with a Vector Field Histogram (VFH) pre-filter. This creates a robust pipeline: the VFH steers the desired heading away from dense obstacles, and the DWB algorithm subsequently samples and evaluates kinematically feasible trajectories against this dynamically adjusted heading.

6.1.1 Environment and Local Map Representation

The robot operates in an unknown environment and does not rely on a pre-computed global SLAM costmap. Instead, the "map" is an instantaneous, localized point cloud generated directly from the live `/scan` topic. Every time a `LaserScan` message is received, the `_scan_to_obstacles` method filters out invalid readings and converts the polar distance measurements into Cartesian (x, y) coordinates in the global odometry frame using the robot's current pose.

Listing 1: Converting LaserScan to Local Cartesian Obstacles

```
1 def _scan_to_obstacles(self):
2     obs = []
3     cy, sy = math.cos(self.state.yaw), math.sin(self.state.
4         yaw)
5     for i, r in enumerate(self.scan_ranges):
6         if not math.isfinite(r) or r <= 0.08 or r > 3.5:
7             continue
8         angle = self.scan_angle_min + i*self.scan_angle_inc
9         lx, ly = r*math.cos(angle), r*math.sin(angle)
10        obs.append((self.state.x+cy*lx-sy*ly, self.state.y+
11            sy*lx+cy*ly))
12    return obs
```

6.1.2 Vector Field Histogram (VFH) Steering

Before the DWB calculates its trajectories, the VFH algorithm processes the 360-degree LiDAR data to build a polar obstacle density histogram (binned at 5-degree intervals). Readings are weighted by the inverse square of their distance, causing nearby obstacles to exert a higher "blockage" penalty. The algorithm determines the blockage ratio of the direct goal vector. If the path is blocked, it searches adjacent histogram bins for the nearest "open" sector that falls below a defined density threshold. Finally, it applies a sigmoid-like blend between the raw goal vector and the open sector's vector. This produces

a "steered vector" that naturally curves around walls while maintaining a pull toward the operator's desired heading.

6.1.3 Zone-Based Dynamic Limits

To ensure safety at high speeds, the node continuously evaluates the distance to the absolute nearest obstacle and categorizes the environment into discrete safety zones: `ZONE_DANGER`, `ZONE_CAUTION`, `ZONE_SLOW`, and `CLEAR`. As the robot transitions into tighter zones, the control loop dynamically scales down the maximum allowable linear speed (`eff_max_speed`) and reduces the goal alignment weight (`eff_align_w`), forcing the robot to prioritize clearance over goal-seeking in cramped quarters.

6.1.4 Algorithm Pseudocode

The core execution loop ties the VFH steering, zone limits, and DWB trajectory rollouts together into a single continuous cycle.

```

1: Input: State  $S(x, y, \theta, v, \omega)$ , Raw Goal Vector  $V_{raw}$ , Obstacles  $O$ , Nearest
   distance  $d_{min}$ 
2: Output: Optimal Command  $(v^*, \omega^*)$ 
3:  $V_{steered}, blockage \leftarrow VFH.steer(S, V_{raw}, O)$ 
4:  $v_{limit}, w_{align} \leftarrow CalculateSafetyZones(d_{min})$ 
5: Compute Dynamic Window  $V_r = [v_{min}, v_{limit}] \times [\omega_{min}, \omega_{max}]$ 
6:  $C_{best} \leftarrow \infty$ 
7:  $Traj_{best} \leftarrow \emptyset$ 
8: for each  $v \in V_r$  and  $\omega \in V_r$  do
9:    $Traj \leftarrow ForwardSimulate(S, v, \omega, predict\_time)$ 
10:   $C_{clearance}, isValid \leftarrow ObstacleCritic(Traj, O)$ 
11:  if not isValid then
12:    continue {Discard colliding trajectories}
13:  end if
14:   $C_{heading} \leftarrow GoalAlignCritic(Traj, V_{steered})$ 
15:   $C_{vel} \leftarrow VelocityCritic(v)$ 
16:   $C_{total} \leftarrow (w_{align} \cdot C_{heading}) + (\beta \cdot C_{clearance}) + (\gamma \cdot C_{vel})$ 
17:  if  $C_{total} < C_{best}$  then
18:     $C_{best} \leftarrow C_{total}$ 
19:     $(v^*, \omega^*) \leftarrow (v, \omega)$ 
20:     $Traj_{best} \leftarrow Traj$ 
21:  end if
22: end for
23: if  $Traj_{best}$  is  $\emptyset$  then
24:   $(v^*, \omega^*) \leftarrow ExecuteEscapeManeuver(V_{steered})$ 
25: end if
26: return  $(v^*, \omega^*)$ 

```

6.1.5 Modular Critic Implementation and Optimization

The trajectory scoring system utilizes modular, independent critic classes. A significant computational optimization is implemented within the `GoalAlignCritic`. Rather than utilizing expensive trigonometric functions (like `atan2`) to calculate the angular difference between the simulated trajectory and the steered vector, the critic computes the normalized dot product of the trajectory's displacement vector and the target vector.

Listing 2: Optimized Goal Alignment Critic via Dot Product

```
1 class GoalAlignCritic:
2     """Score trajectory by displacement alignment with a
3     world-frame unit vector."""
4     def score(self, traj, gv_x, gv_y):
5         if not traj or len(traj) < 2:
6             return 1.0
7         dx = traj[-1][0] - traj[0][0]
8         dy = traj[-1][1] - traj[0][1]
9         dist = math.hypot(dx, dy)
10        if dist < 1e-6:
11            return 1.0
12
13        # Calculate dot product for highly efficient angular
14        # difference scoring
15        dot = (dx/dist)*gv_x + (dy/dist)*gv_y
16        return (1.0 - dot) / 2.0 # 0 = perfect alignment,
17                                # 1 = opposite direction
```

6.1.6 Fallback Recovery and Visualisation

In highly cluttered environments, it is possible for the `ObstacleClearanceCritic` to reject all sampled trajectories due to proximity thresholds, resulting in an empty valid trajectory list. When this "no valid trajectory" condition is met, the node executes a fallback escape maneuver. The robot sets its linear velocity to strictly zero and spins in place, tapering its angular velocity proportionally to the angular error between its current heading and the VFH-steered open vector. For real-time operational transparency, the `_publish_viz` method leverages `visualization_msgs/MarkerArray` to project the DWB internal state into RViz. All evaluated candidate trajectories are rendered as a translucent blue fan, while the ultimately selected optimal trajectory is highlighted in bright green, allowing for immediate visual verification of the critic scoring outcomes.

6.2 MPPI Algorithm

Similar to the DWB implementation, the implemented Model Predictive Path Integral (MPPI) algorithm operates as a standalone node (`MPPIPlannerNode`) and reacts instantaneously to the local environment.

6.2.1 Environment and Local Map Representation

The MPPI node shares identical preprocessing steps to the DWB approach when constructing the instantaneous perception map. The live **LaserScan** arrays are filtered for validity and resolved into robot-centric Cartesian coordinates, discarding infinite values to immediately serve as obstacle clusters for the sampling environment.

6.2.2 Algorithm Pseudocode and Kinematic Simulation

The internal MPPI core samples $K = 100$ random control noise sequences over a discrete time horizon T , injecting them into a nominal control sequence. It simulates these kinematic sequences forward using a first-order Euler approximation ($x_{t+1} = x_t + v \cos(\theta) \Delta t$).

- 1: **Input:** Current State $S(x, y, \theta)$, Obstacles O , Nominal Sequences U_V, U_W
- 2: **Output:** Optimal Immediate Command (v^*, ω^*)
- 3: **Parameters:** Samples $K = 100$, Variance $\Sigma = (\sigma_v, \sigma_\omega)$
- 4: Sample K noise sequences $\Delta V, \Delta W \sim \mathcal{N}(0, \Sigma)$ over horizon T
- 5: **for** $k = 1$ to K **do**
- 6: $Traj_k \leftarrow \{S\}$
- 7: $cost_k \leftarrow 0$
- 8: **for** $t = 1$ to T **do**
- 9: Compute step controls $v = U_{V,t} + \Delta V_{k,t}, \omega = U_{W,t} + \Delta W_{k,t}$
- 10: Clamp (v, ω) to safe hardware limits $\rightarrow [v_{min}, v_{max}] \times [\omega_{min}, \omega_{max}]$
- 11: $S \leftarrow \text{UnicycleForwardEulerSim}(S, v, \omega, \Delta t)$
- 12: Append S to $Traj_k$
- 13: **end for**
- 14: $cost_k \leftarrow \text{EvaluateCost}(Traj_k, O, \text{Goal})$
- 15: **end for**
- 16: $min_cost \leftarrow \min(cost_1 \dots cost_K)$
- 17: Calculate weights: $w_k = \exp\left(-\frac{cost_k - min_cost}{\lambda}\right)$
- 18: Normalize weights: $w_k \leftarrow \frac{w_k}{\sum w_k}$
- 19: Update sequences: $U_V \leftarrow U_V + \sum_{k=1}^K w_k \Delta V_k \quad | \quad U_W \leftarrow U_W + \sum_{k=1}^K w_k \Delta W_k$
- 20: Shift nominal sequences forward in time by one step
- 21: **return** First action of sequences $(U_{V,0}, U_{W,0})$

6.2.3 Trajectory Evaluation, Critic Implementation, and Complexity

MPPI assigns compound costs over the entirety of the simulated sequences. The evaluation applies penalties for structural collisions with the `robot_radius`, preventing the selection of globally dangerous trajectories.

Clear sequences receive an accumulated proportional penalty evaluating heading offset, clearance distance, and deviation from the optimal maximum speed.

Note: It should be noted that the current programmatic evaluation relies on standard nested iteration logic over K samples, T timesteps, and N obstacle scan points, yielding an $O(K \times T \times N)$ computational complexity. While adequate for early prototyping, matrix-vectorized operations (eg. *we have used NumPy*) or GPU tensor computations are typically necessary to guarantee high real-time execution frequencies under dense dynamic obstacle conditions without blocking Python’s primary thread.

Listing 3: MPPI Stochastic Trajectory Evaluation

```

1      # Collision check
2      for (px, py, pyaw) in traj:
3          for (ox, oy) in obstacles:
4              d = math.hypot(px - ox, py - oy)
5              if d < r_robot:
6                  cost_k += 10000.0 # High collision
6                      penalty (fatal)
7              if d < min_dist:
8                  min_dist = d
9
10     # Clearance cost
11     if min_dist <= r_penalty and min_dist > r_robot:
12         cost_k += p('dist_cost_gain') * (1.0 - (
13             min_dist / r_penalty))
14
15     # Heading cost
16     final_yaw = traj[-1][2]
17     heading_diff = abs(math.atan2(math.sin(
18         desired_yaw - final_yaw), math.cos(
19         desired_yaw - final_yaw)))
20     cost_k += p('heading_cost_gain') * (heading_diff
21         / math.pi)
22
23     # Velocity cost (average speed diff)
24     avg_v = np.mean(V_samples[k, :])
25     if p('max_speed') > 0:
26         cost_k += p('velocity_cost_gain') * (abs(
27             target_v - avg_v) / p('max_speed'))

```

6.2.4 Control Update and Execution

Once the trajectory state costs C_k are formulated, the algorithm transitions into a control update utilizing a Softmax operation parameterized by temperature λ ('lam = 0.5'). Lower-cost trajectories heavily dominate the resultant weighted summation, dragging the nominal velocity control profile toward successful, collision-free maneuvers.

Listing 4: MPPI Information-Theoretic Control Update

```

1      # Compute weights
2      lam = 0.5
3      min_cost = np.min(costs)
4      weights = np.exp(-(costs - min_cost) / lam)
5      weights /= np.sum(weights)
6
7      # Update control sequence with weighted noise
8      self.u_V += np.sum(weights[:, None] * delta_V, axis
9                          =0)
10     self.u_W += np.sum(weights[:, None] * delta_W, axis
11                        =0)
12
13     # Clip again to hardware config margins
14     self.u_V = np.clip(self.u_V, p('min_speed'), p('
15     max_speed'))
16     self.u_W = np.clip(self.u_W, -p('max_yaw_rate'), p('
17     max_yaw_rate'))
18
19     # Best command is the first element
20     cmd_v = self.u_V[0]
21     cmd_w = self.u_W[0]

```

6.2.5 ROS 2 Visualisation

To inspect the sampled distributions during deployment, MPPI broadcasts the immense "tree" of 100 evaluated random trajectory rollouts via a translucent cyan `MarkerArray` published on `/mppi_planner/trajectories`. The optimal trajectory representing the updated sequence is concurrently published in bold red over `/mppi_planner/best_trajectory` for immediate RViz comparison against the DWB candidates.

7 Simulations

7.1 DWB

7.1.1 Static Environment

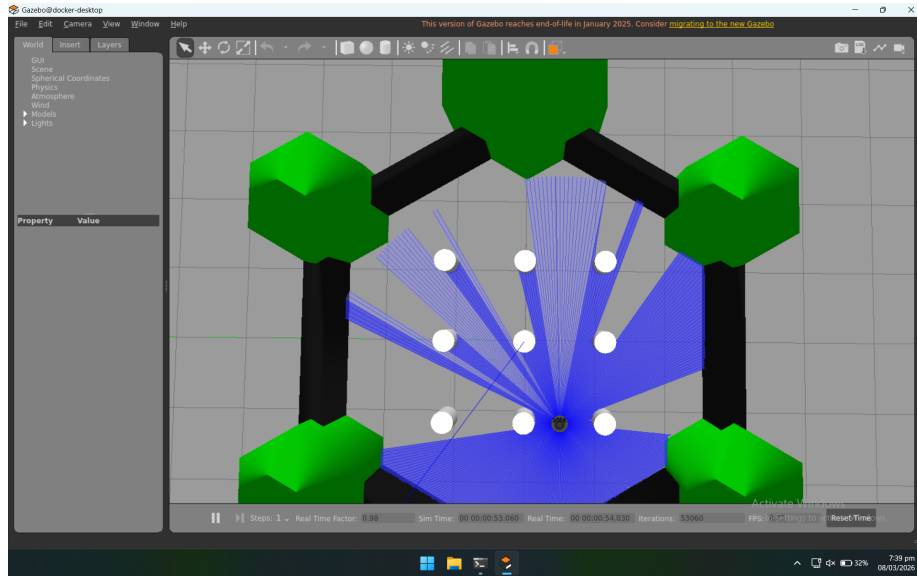


Figure 1: Turtlebot3 in Gazebo for the DWB Algorithm in a static environment

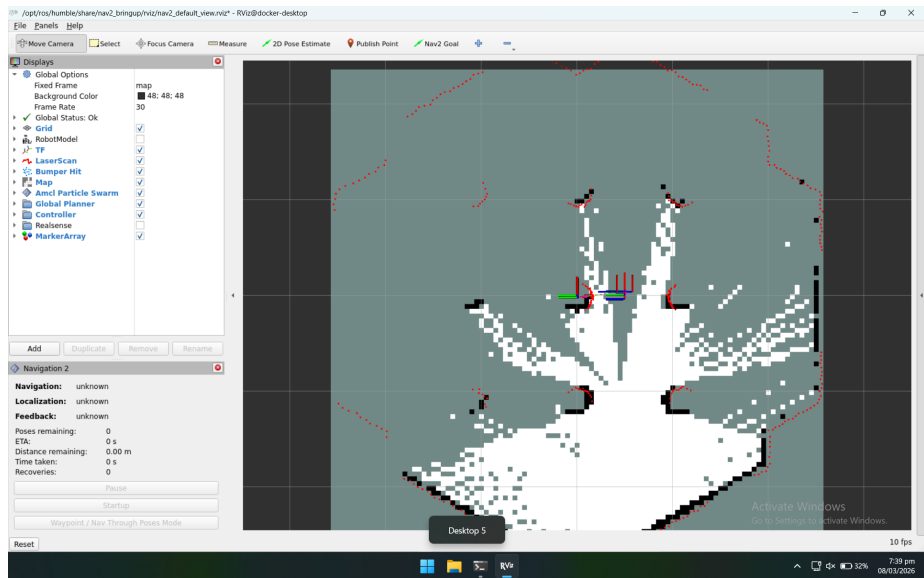


Figure 2: Turtlebot3 in RViz for the DWB Algorithm in a static environment

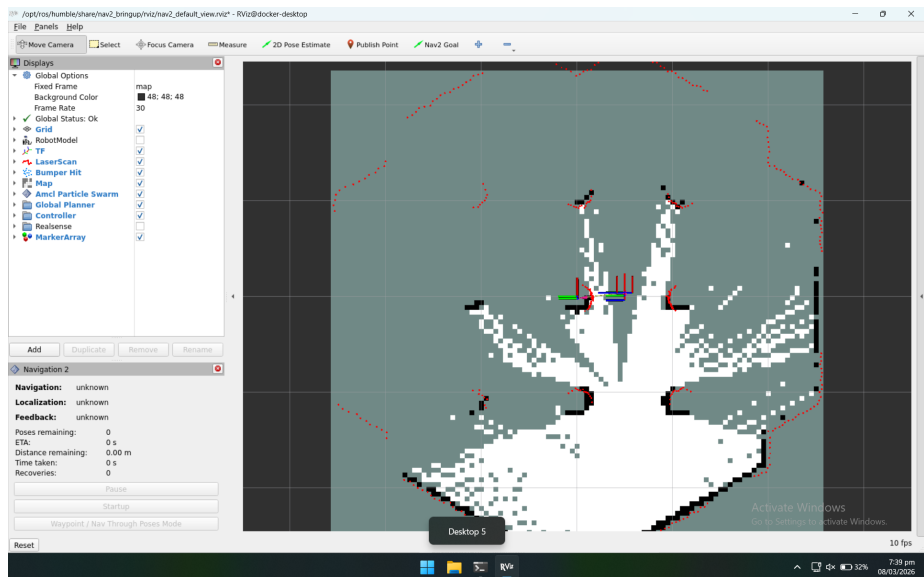


Figure 3: Turtlebot3 in RViz for the DWB Algorithm in a static environment

Here is a video showing our implementation of the DWB+VFH algorithm on a map of static obstacles : https://iiithydstudents-my.sharepoint.com/:v:/g/personal/bejaine_birju_students_iiit_ac_in/IQBqcNSkmJqtSp-0jI58QkL5AVqstgJX_052kyEWn01Ibs0?nav=eyJyZWZlcnJhbEluZm8iOj0nsicmVmZXJyYWxBcHAiOiJPbmVEcm12ZUZvckJ1c2luZXNzIiwicmVmZXJyYWxBcHBQbGF0Zm9ybSI6IldlYiIsInJlZmVycmFsTW9kZSI6InZpZXciLCJyZWZlcnJhbFZpZXciOiJNeUZpbGVzTGlua0NvcHkifX0&e=54jMoJ

7.1.2 Dynamic Environment

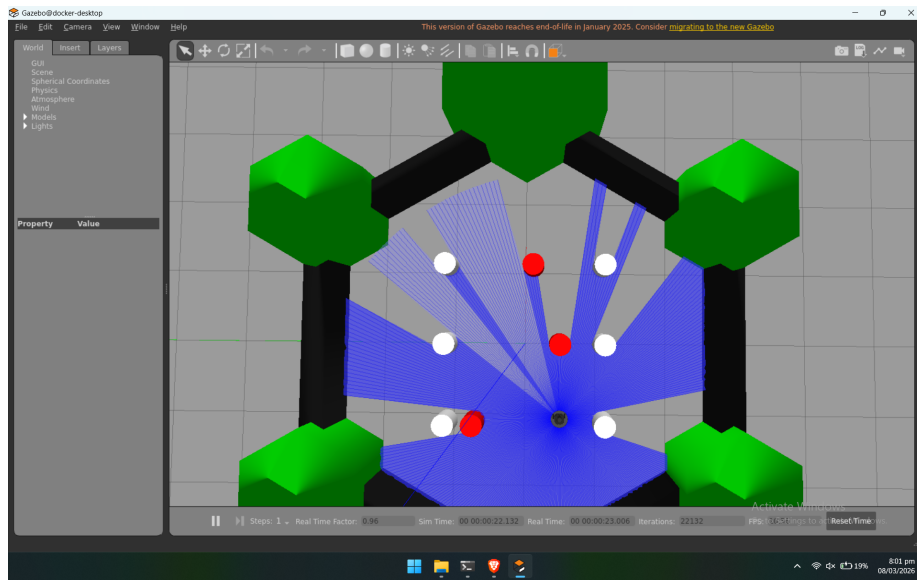


Figure 4: Turtlebot3 in Gazebo for the DWB Algorithm in a dynamic environment

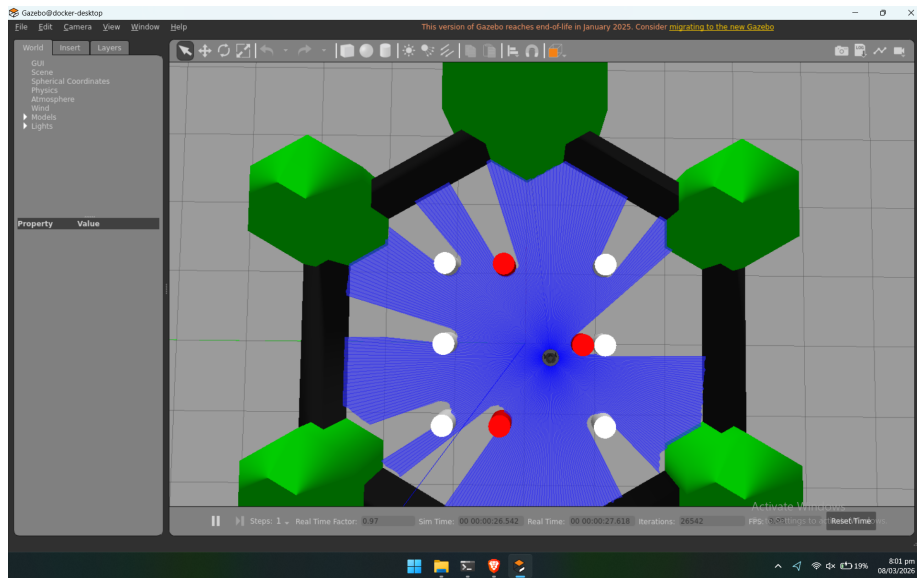


Figure 5: Turtlebot3 in Gazebo for the DWB Algorithm in a dynamic environment

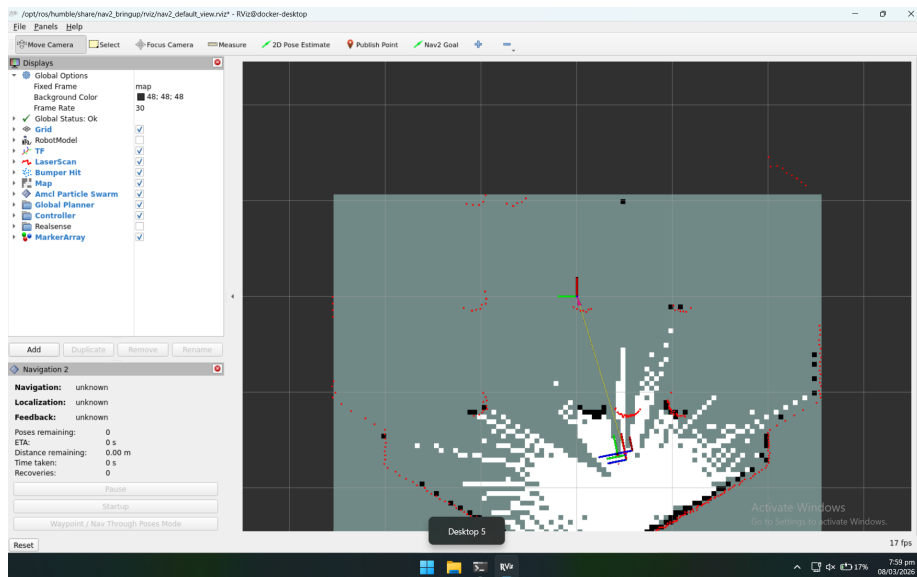


Figure 6: Turtlebot3 in RViz for the DWB Algorithm in a dynamic environment

Here is a video showing our implementation of the DWB+VFH algorithm on a map of dynamic obstacles : https://iiithdstudents-my.sharepoint.com/:v:/g/personal/bejaine_birju_students_iiit_ac_in/IQCSFScv22XBSYoCwTBGWcDtARvtiK2PzcIawzUn2KC1dkY?nav=eyJyZWZlcnJhbEluZm8iOj0nsicmVmZXJyYWxBcHAiOiJPbmVEcm12ZUZvckJ1c2luZXNzIiwicmVmZXJyYWxBcHBQbGF0Zm9ybSI6IldldlYiIsInJlZmVycmFsTW9kZSI6InZpZXciLCJyZWZlcnJhbFZpZXciOiJNeUZpbGVzTGlua0NvcHkifX0&e=wIRhSW

7.1.3 Realistic Environment

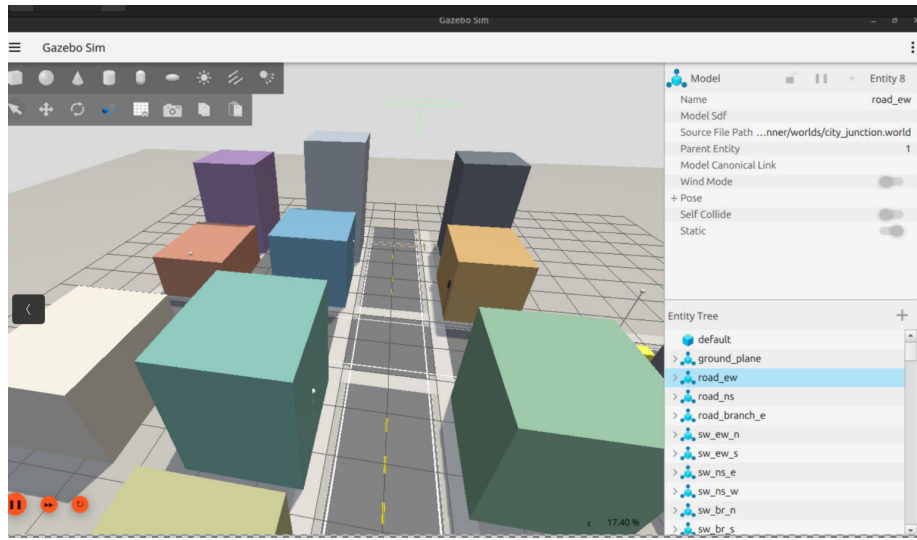


Figure 7: TurtleBot3 in the cityscape map avoiding static and dynamic obstacles in Gazebo

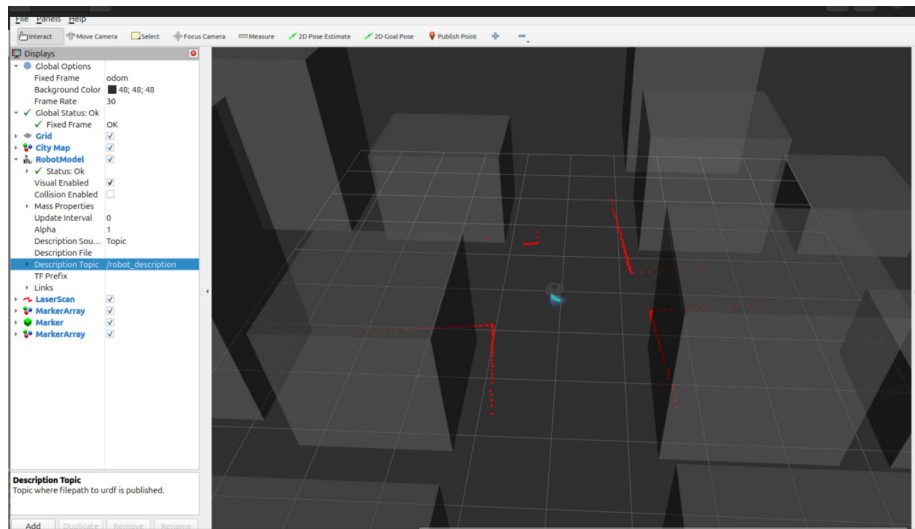


Figure 8: TurtleBot3 in the cityscape map avoiding static and dynamic obstacles in RViz

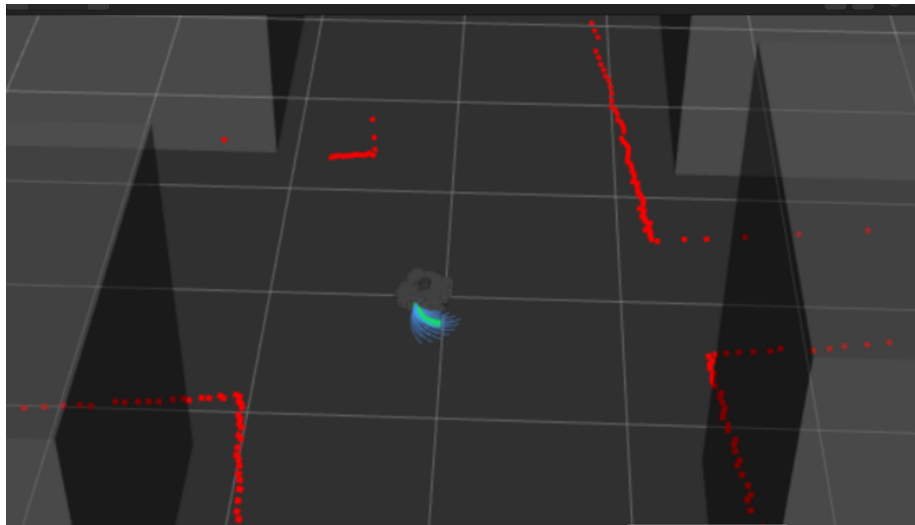


Figure 9: TurtleBot3 in the cityscape map avoiding static and dynamic obstacles in RViz

Here is a video showing our implementation of the DWB+VFH algorithm on a realistic map with both static and dynamic obstacles : <https://drive.google.com/file/d/1QcucTuRu0YR4Np4chcifDGTzEx4qBP3Q/view?usp=sharing>

7.2 MPPI

7.2.1 Static Environment

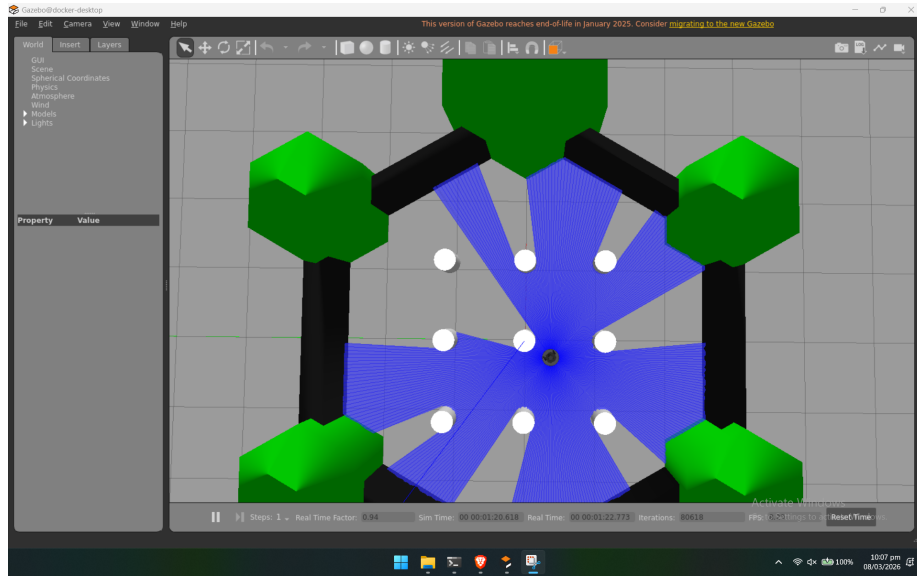


Figure 10: Turtlebot3 in Gazebo for the MPPI Algorithm in a static environment

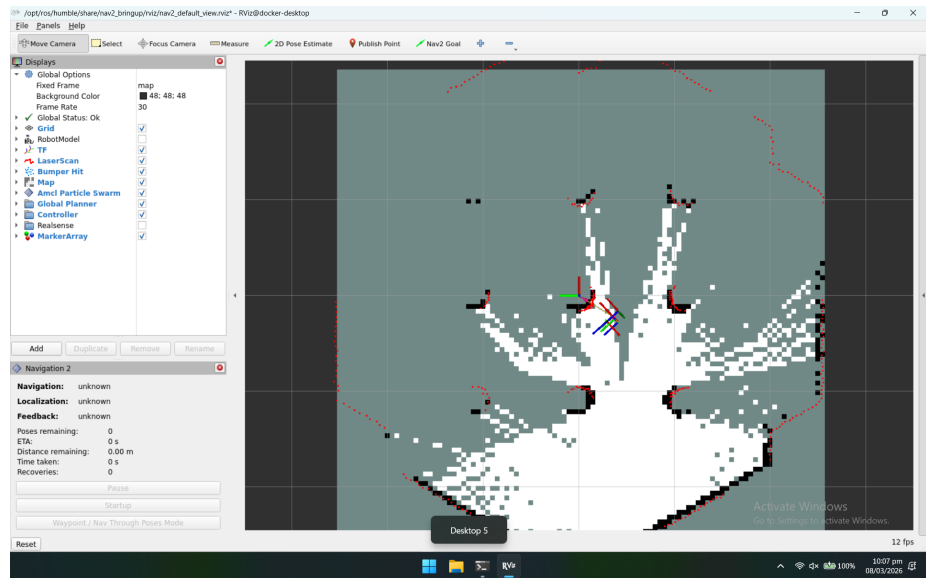


Figure 11: Turtlebot3 in RViz for the MPPI Algorithm in a static environment

7.2.2 Dynamic Environment

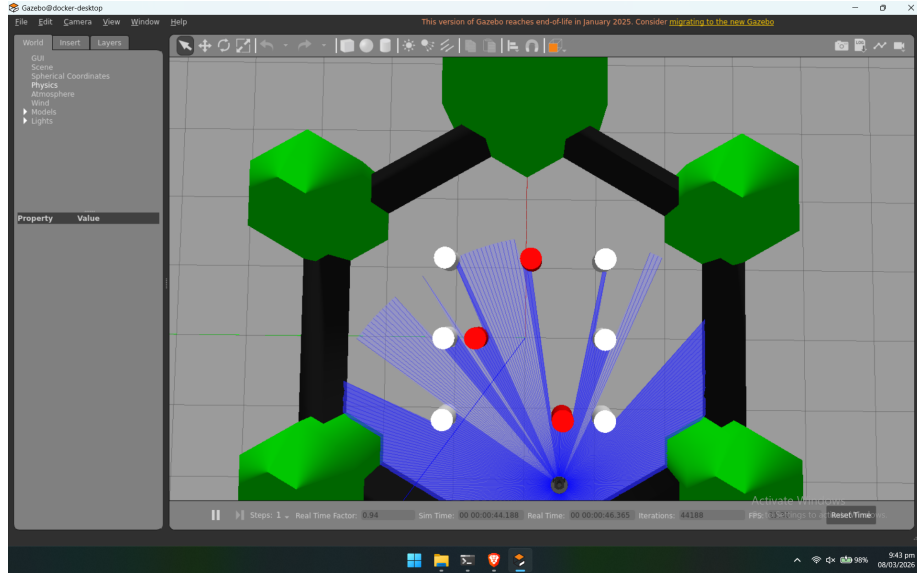


Figure 12: Turtlebot3 in Gazebo for the MPPI Algorithm in a dynamic environment

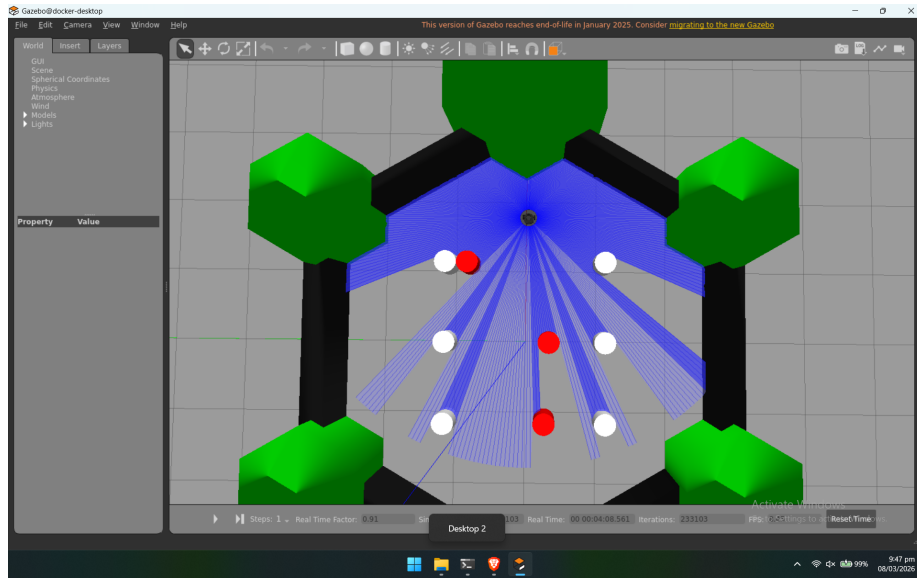


Figure 13: Turtlebot3 in Gazebo for the MPPI Algorithm in a dynamic environment

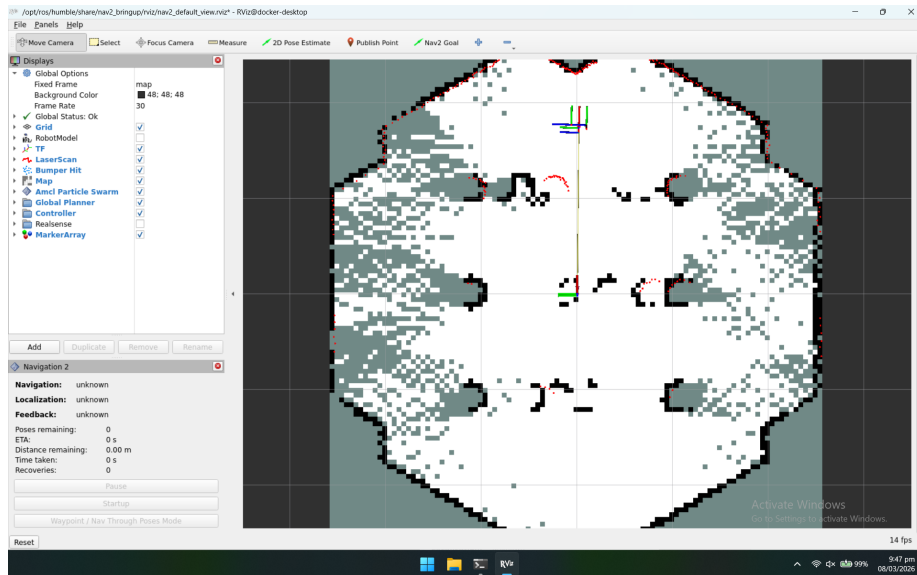


Figure 14: Turtlebot3 in RViz for the MPPI Algorithm in a dynamic environment

7.2.3 Realistic Environment

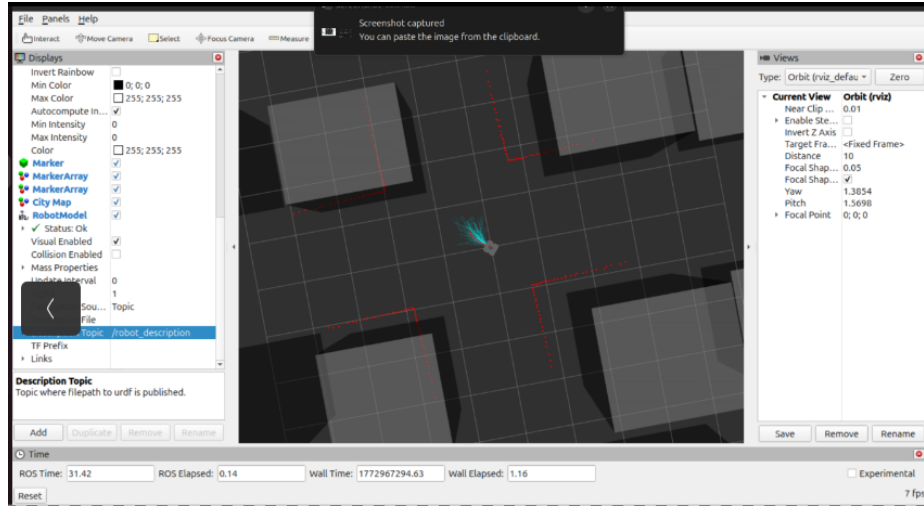


Figure 15: TurtleBot3 in the realistic map avoiding static and dynamic obstacles in RViz

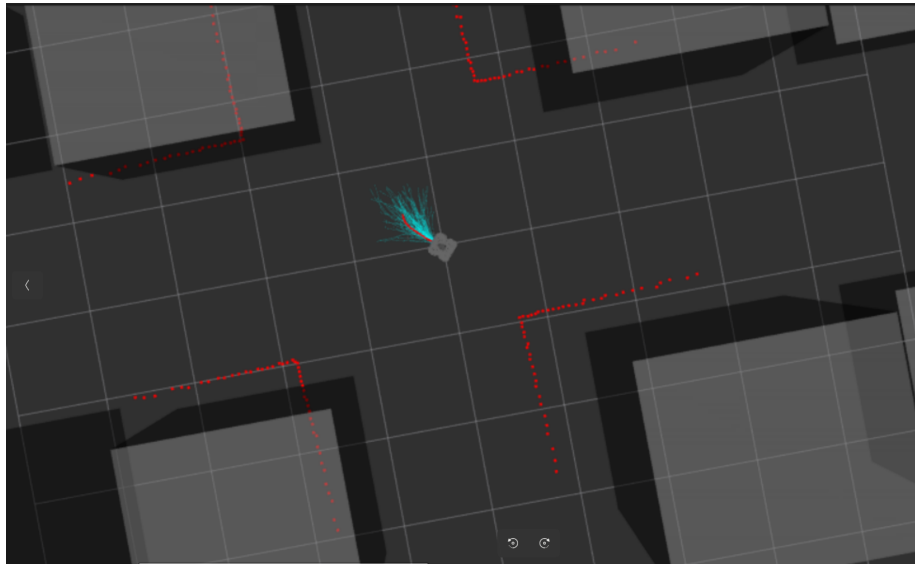


Figure 16: TurtleBot3 in the realistic map avoiding static and dynamic obstacles in RViz

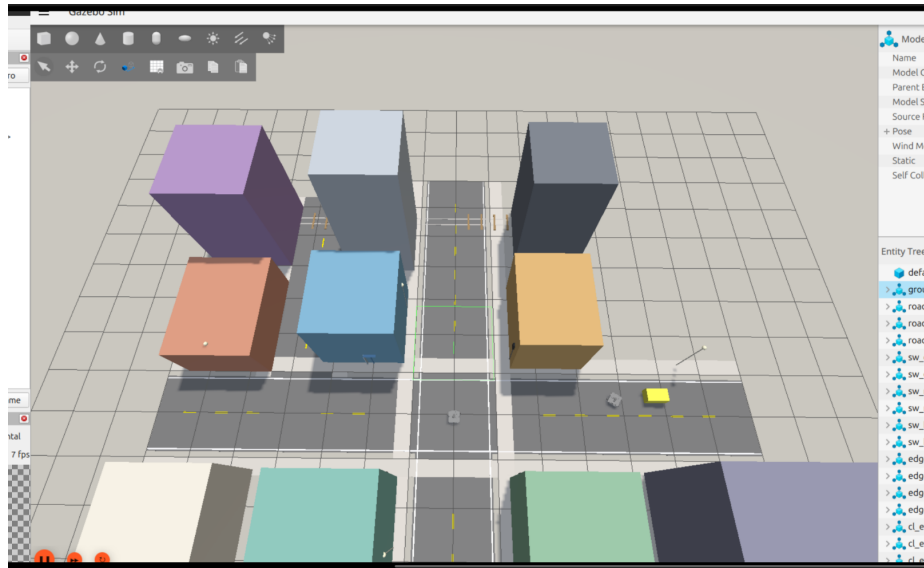


Figure 17: TurtleBot3 in the realistic map avoiding static and dynamic obstacles in Gazebo

Here is a video showing our implementation of the MPPI algorithm on a realistic map with both static and dynamic obstacles : <https://drive.google.com/file/d/1HG60WtVu3K8fJt9BD9PAIX7ly95Px9UV/view?usp=sharing>

8 Performance Analysis

To rigorously evaluate and compare the performance of the Dynamic Window Based (DWB) controller (augmented with a Vector Field Histogram pre filter) and the Model Predictive Path Integral (MPPI) algorithm, we conducted a series of simulation experiments. The evaluation focuses on the algorithms’ ability to navigate safely and efficiently in an unknown environment using instantaneous directional commands (velocity vectors) rather than pre computed global paths.

8.1 Experimental Setup and Environments

To thoroughly assess both static obstacle avoidance and dynamic anticipatory behaviors, the tests were conducted across two distinct simulated environments:

- **Static Environment** : A map populated exclusively with stationary obstacles (walls, boxes, and cylinders) to test tight space navigation, cornering, and basic clearance.
- **Dynamic Environment** : A map featuring moving obstacles (such as sliding cylinders) to test the reactive limits of DWB and the predictive capabilities of MPPI.

8.2 Evaluation Metrics

A custom asynchronous ROS 2 telemetry node (`metrics_logger`) was developed to record the robot’s performance continuously during each run. Because the robot operates on open ended directional vectors rather than fixed (x, y) goal coordinates, traditional metrics like “Distance to Goal” were replaced with metrics that evaluate the “Quality of the Journey”. The node tracks the following quantitative metrics from the moment a directional command is issued until the robot is commanded to stop:

- **Total Distance (m)** : The cumulative path length physically traversed by the robot.
- **Total Time (s)** : The elapsed time during the active navigation phase.
- **Average Speed (m/s)** : Evaluates the algorithm’s ability to maintain forward momentum without excessive deceleration or hesitation.
- **Minimum Clearance (m)** : The absolute closest proximity to any obstacle during the run, serving as the primary safety and collision avoidance metric.
- **Path Smoothness (Angular Variance)**: Calculated as the variance in the robot’s angular velocity over the run. A lower variance indicates a smooth, continuous trajectory, whereas a high variance indicates oscillatory, jittery, or “ping ponging” behavior.

- **Compute Load (CPU %)** : The average system processor utilization, allowing us to quantify the hardware cost of MPPI’s stochastic tensor operations against DWB’s discrete velocity sampling.

8.3 Observation

Both algorithms were subjected to runs across the static and dynamic maps. The recorded data is summarized in Table 3.

Here are the images showing the metrics logger node, logging all the measured parameters :

```

[INFO] [1772978733.237365815] [performance_monitor]: — RUN STARTED: Recording Metrics —
[INFO] [1772978763.037413719] [performance_monitor]: — RUN FINISHED: Metrics Saved —
[INFO] [1772978763.038184884] [performance_monitor]: Time: 29.80s | Dist: 4.62m | Speed: 0.15m/s
[INFO] [1772978763.038846004] [performance_monitor]: Clearance: 0.25m | Smoothness: 0.6898 | CPU: 21.6%

[INFO] [1772987929.553255796] [performance_monitor]: — RUN STARTED: Recording Metrics —
[INFO] [1772987985.126922418] [performance_monitor]: — RUN FINISHED: Metrics Saved —
[INFO] [1772987985.127633074] [performance_monitor]: Time: 55.57s | Dist: 7.14m | Speed: 0.13m/s
[INFO] [1772987985.128202275] [performance_monitor]: Clearance: 0.18m | Smoothness: 0.1683 | CPU: 23.8%

[INFO] [1772980752.443784683] [performance_monitor]: — RUN STARTED: Recording Metrics —
[INFO] [1772980778.741730829] [performance_monitor]: — RUN FINISHED: Metrics Saved —
[INFO] [1772980778.742347818] [performance_monitor]: Time: 26.30s | Dist: 4.24m | Speed: 0.16m/s
[INFO] [1772980778.742812954] [performance_monitor]: Clearance: 0.26m | Smoothness: 0.1435 | CPU: 22.4%

[INFO] [1772986521.437024222] [performance_monitor]: — RUN STARTED: Recording Metrics —
[INFO] [1772986565.936505255] [performance_monitor]: — RUN FINISHED: Metrics Saved —
[INFO] [1772986565.937279839] [performance_monitor]: Time: 44.50s | Dist: 4.10m | Speed: 0.09m/s
[INFO] [1772986565.937788323] [performance_monitor]: Clearance: 0.43m | Smoothness: 0.2365 | CPU: 25.9%

```

Figure 18: Measured parameters on the terminal

Metric	Static Environment		Dynamic Environment	
	DWB + VFH	MPPI	DWB + VFH	MPPI
Total Time (s)	29.80	55.57	26.30	44.50
Distance (m)	4.62	7.14	4.24	4.10
Avg Speed (m/s)	0.15	0.13	0.16	0.09
Min Clearance (m)	0.25	0.18	0.26	0.43
Smoothness	0.6898	0.1683	0.1435	0.2365
CPU Load (%)	21.6	23.8	22.4	25.9

Table 3: Performance metrics comparison between DWB and MPPI algorithms.

8.4 Observations and Analysis

8.4.1 Static Environment Navigation

In the static environment, DWB+VFH proved highly efficient in terms of time (29.80s) and distance (4.62m), acting as a direct and aggressive planner. However, this efficiency came at the cost of trajectory smoothness. The angular variance for DWB spiked to 0.6898. This occurs because the VFH pre-filter utilizes discrete 5-degree bins; when the robot navigates around tight static corners

(like the hexagonal map obstacles), the "open" bin shifts abruptly, causing the DWB to apply sharp, jittery steering corrections.

Conversely, MPPI took significantly longer (55.57s) and covered more distance (7.14m) but maintained an exceptionally smooth trajectory (angular variance of 0.1683). By optimizing over a forward-simulated horizon of 1.5 seconds and applying softmax weighting across 100 trajectories, MPPI naturally executes wide, sweeping arcs rather than sharp reactive turns, resulting in a much more kinetically stable motion profile.

8.4.2 Dynamic Environment Navigation

The dynamic environment highlighted the fundamental difference between reactive and predictive planning. DWB+VFH maintained a high average speed (0.16 m/s) and completed the course rapidly (26.30s). Because DWB only evaluates the immediate time step, it does not decelerate for moving obstacles until they physically encroach on its safety radius, forcing rapid, last-second evasions. MPPI exhibited strong anticipatory behavior. Observing the dynamic trajectories, MPPI's average speed plummeted to 0.09 m/s, and its run time extended to 44.50s. The algorithm's multi-step horizon detected moving pillars intersecting its future path, causing the robot to proactively decelerate or yield, waiting for the obstacle to pass before proceeding. This predictive yielding resulted in a massive safety clearance of 0.43m, compared to DWB's 0.26m.

8.4.3 Computational Load

As expected, MPPI consistently demanded higher CPU utilization (up to 25.9%) compared to DWB (21.6% - 22.4%). Simulating 100 kinematic rollouts over a multi-step horizon is mathematically heavier than evaluating a single dynamic window. However, the use of vectorized tensor operations via `numpy` successfully mitigated this cost, allowing the high-frequency control loop to run flawlessly on standard consumer hardware without latency degradation.

8.5 Final Decree

Both algorithms successfully fulfilled the requirements of the local path planning problem statement, but they excel in entirely different paradigms:

- **DWB + VFH** is the superior choice for *static, highly-constrained environments*. Its myopic, reactive nature allows it to squeeze through tight spaces and take direct paths efficiently.
- **MPPI** is the superior choice for *dynamic, unstructured environments*. Its ability to foresee moving obstacles, yield proactively, and carve exceptionally smooth, continuous trajectories provides a massive safety advantage, easily justifying its slightly higher computational cost and slower traversal times.

References

- [1] S. Elbouhy, A. Adamanov, P. Braun, and H. Rose, “Comparative analysis of local trajectory planning algorithms in ros2,” in *Proceedings of SIMPAR*, April 2025, pp. 1–6.
- [2] ROBOTIS Co., Ltd., “Turtlebot3 simulation e-manual.” [Online]. Available: <https://emanual.robotis.com/docs/en/platform/turtlebot3/simulation/>
- [3] X. Gong, Y. Gao, F. Wang, D. Zhu, W. Zhao, F. Wang, and Y. Liu, “A local path planning algorithm for robots based on improved dwa,” *Electronics*, vol. 13, no. 15, p. 2965, 2024. [Online]. Available: <https://www.mdpi.com/2079-9292/13/15/2965>
- [4] O. Khatib, “Real-time obstacle avoidance for manipulators and mobile robots,” in *Proceedings. 1985 IEEE International Conference on Robotics and Automation*, vol. 2. IEEE, 1985, pp. 500–505.
- [5] F. Peralta, M. Arzamendia, D. Gregor, D. G. Reina, and S. Toral, “A comparison of local path planning techniques of autonomous surface vehicles for monitoring applications: The ypacarai lake case-study,” *Sensors*, vol. 20, no. 5, p. 1488, 2020. [Online]. Available: <https://pmc.ncbi.nlm.nih.gov/articles/PMC7085648/>
- [6] D. Gomez-Ayalde and V. A. Romero-Cano, “Local planning methods for autonomous navigation on sidewalks: a comparative survey,” Robotics and Autonomous Systems Laboratory, Universidad Autonoma de Occidente. [Online]. Available: <https://orca.cardiff.ac.uk/id/eprint/167063/7/Local%20planning%20methods%20for%20autonomous%20navigation%20on%20sidewalks%20-%20a%20comparative%20survey.pdf>
- [7] D. Fox, W. Burgard, and S. Thrun, “The dynamic window approach to collision avoidance,” *IEEE Robotics & Automation Magazine*, vol. 4, no. 1, pp. 23–33, 1997. [Online]. Available: https://www.ri.cmu.edu/pub_files/pub1/fox_dieter_1997_1/fox_dieter_1997_1.pdf